FDR Portal (SIH26265): Problem Report and Winning Strategy

Problem statement: *Streamline of Security Deposit / FDR Portal* (AICTE, Smart Automation theme)

1. The problem in plain words

AICTE approves technical institutions. For some approvals, an institution must lock up money with a bank as a guarantee, either as a **security deposit** or a **Fixed Deposit Receipt (FDR)**. That money is held until the institution has met its obligations, after which it is released or refunded.

AICTE's Regulation Bureau therefore has to track, for every institution, one or more financial instruments: who issued them, how much, when they were issued, when they mature, whether they are genuine, whether they need renewal, and whether a release request is valid.

Today this is done through scattered records, manual correspondence and multiple tracking mechanisms. The result is that nobody has a reliable, current view of which deposits are healthy, which are about to lapse, which are unverified, and which are waiting on an action.

The ask is a **single secure portal** that manages the whole lifecycle of each deposit, reminds people before things go wrong, and keeps a trustworthy record of everything.

2. The lifecycle the portal must cover

1. **Submission.** The institution submits the instrument details and supporting documents (FDR copy, deposit confirmation and so on).
2. **Scrutiny and verification.** An AICTE official reviews the record, flags missing or inconsistent items, and verifies the instrument through bank-issued verification where that is supported.
3. **Monitoring.** The system tracks validity and maturity dates and sends configurable reminders before deadlines. Overdue or unresolved cases are flagged.
4. **Renewal or replacement.** The institution submits a renewed or replacement instrument, and the system keeps a complete history of changes.

5. **Release, discharge or refund.** The institution requests release. The request is processed only if prescribed conditions are met and the competent authority approves.
6. **Reporting and audit.** Dashboards and reports give an overview, and every action is traceable.

3. What the statement explicitly asks for

The ten expected capabilities are the checklist judges will most likely score against:

#	Requirement	What it means in practice
1	Centralized repository	One place holding institution-wise and instrument-wise records, documents and status
2	Digital document management	Secure upload, storage, retrieval and review of FDRs and confirmations
3	Verification and discrepancy detection	Check through authorized sources where available; flag missing data, inconsistencies, documents needing scrutiny
4	Automated validity and maturity tracking	Watch dates and send advance reminders
5	Renewal and replacement monitoring	Track renewed instruments and keep full change history
6	Release and refund workflow	Submission, processing and status tracking of release requests
7	Automated alerts and escalation	Notify about pending verification, approaching deadlines, overdue actions, unresolved cases
8	Dashboard and management reports	Active, expiring, matured, renewed, released and pending views
9	Audit trail and access control	Traceable record of submissions, verification, changes, approvals and actions

#	Requirement	What it means in practice
10	Integration and data security	Support authorized integration; protect confidential information

4. Hidden signals in the wording

These phrases tell you what the problem-setters care about beyond the list:

- **"Distinguish between information submitted by an institution and information independently confirmed by an authorized bank."** This is a deliberate trust requirement. It is the single most important idea in the statement.
- **"Wherever supported" / "where technically feasible and formally authorized."** They expect integrations to be designed for, not necessarily live. Mocking is acceptable if you are honest about it.
- **"Configurable reminders."** Rules must be adjustable, not hard-coded.
- **"Subject to applicable regulations... authorization by the competent authority."** Release must be gated by approvals. Software must support officials, not replace them.
- **"Demonstrate measurable improvements."** They want numbers: record accuracy, verification turnaround, timely renewal monitoring, manual effort reduction, visibility of pending cases.
- **"Testing should cover scenarios such as..."** They name the test cases: new FDR submission, validity tracking, approaching maturity, renewal, discrepancy identification, authorized release. Treat this as the demo script.

5. What most teams will do, and why that is not enough

The typical submission will be an upload form, a table of FDRs, email reminders before expiry and a dashboard with a few charts. It will tick boxes 1, 2, 4 and 8 and be hard to tell apart from the others.

To stand out, you need an organising idea, depth on the parts others skim (verification, escalation, audit, release) and a demo that proves it works.

6. Organising idea: every FDR has a trust level and a clock

Build the product around two questions an official actually asks:

1. **How much of this record is proven, and how much is just claimed?**
2. **What is about to go wrong, and who has to act on it?**

Everything in the product ladders up to one of these.

7. Signature features

7.1 Trust tiers on every field

Every data point carries a badge: *Institution-claimed*, *Document-attached* or *Bank-confirmed*. When tiers disagree (the claimed amount differs from the bank's response), the system raises a discrepancy card showing both values side by side. This directly implements the distinction the statement asks for and makes it visible.

7.2 Mock bank verification service

Build a small separate service that accepts FDR number, bank and amount and returns *confirmed*, *mismatch*, *source unavailable* or *pending*. Store the source, timestamp and result for every call. State clearly that this is a stand-in for authorized bank integrations, designed for plug-in.

7.3 Fast-forward clock

A simulated "today" control for the demo. Jump ahead 45 days and show reminders firing, an FDR flipping to *expiring*, an unresolved case escalating up the officer chain and a lapsed instrument being flagged. A time-based problem is best demonstrated by time passing.

7.4 Instrument passport

One page per FDR with a full timeline of events, attached documents, verification history and the renewal chain (old FDR to replacement FDR). This satisfies the complete-history requirement in the most visible way.

7.5 Risk-ranked "my day" queue

Instead of only counts, give each official a ranked worklist ordered by amount at risk, days to maturity and missing verification. This is what a busy official would use daily, and it shows product thinking beyond the checklist.

7.6 Release workflow with a conditions checklist

Release or refund requests carry the conditions that must be met, a multi-level approval chain, and an automatic block on release while a discrepancy or pending verification is open. This reflects the statement's insistence on competent-authority authorization.

7.7 Tamper-evident audit log

Chain each log entry with a hash of the previous one and show a "log integrity: verified" indicator. It is inexpensive to build and memorable on screen. It supports requirement 9 convincingly.

7.8 Institution self-service view

A separate view for institutions showing what they have on file, what is expiring and a one-click path to upload a renewal. This shows the portal serves both sides, not only officers.

7.9 Configurable rules screen

Let an admin change reminder offsets (for example 90, 60, 30 and 7 days), the escalation ladder and required documents per instrument type. This answers "configurable reminders" and shows the system is not hard-coded.

7.10 Realistic domain detail

Include details an insider would recognise: lien marking, bank confirmation letters, discrepancy reasons such as amount mismatch, wrong beneficiary name or date inconsistency, and a release approval chain with named roles. This signals you understand how the process works in practice.

8. Roles

- **Institution user:** submits instruments, uploads renewals, requests release, views own status.

- **Scrutiny officer:** reviews records, runs verification, records observations, flags discrepancies.
- **Senior approver (competent authority):** approves or rejects release and handles escalations.
- **Admin:** manages users, rules and reminder settings.
- **Auditor (read-only):** views the audit trail and reports.

9. Data model (core entities)

- Institution (id, name, region, contact)
- Instrument (id, institution, type, reference number, issuing bank, amount, issue date, maturity date, status, trust tier per field)
- Document (id, instrument, type, file, uploaded by, timestamp)
- Verification record (id, instrument, source, request, response, result, timestamp)
- Renewal link (old instrument, new instrument)
- Release request (id, instrument, conditions, approvals, status)
- Reminder rule and escalation rule (offset days, recipient role, channel)
- Event log (id, actor, action, before and after values, timestamp, previous hash, hash)
- User and role

10. Architecture outline

- **Frontend:** React with role-based views and dashboard.
- **Backend API:** Node or Python, with role-based access control and an events table.
- **Database:** PostgreSQL.
- **Scheduler:** a job that evaluates rules against the simulated or real date and creates alerts and escalations.
- **Mock bank service:** a separate small API.
- **Storage:** object storage for documents with encryption at rest.
- **Notifications:** in-app plus email (a sandbox or console output is fine for the demo).
- **Security:** authentication, role checks on every endpoint, document access logging, encryption in transit and at rest.

11. Build order

1. Data model, authentication and roles.
2. Instrument submission and document upload.
3. Officer review queue and observations.
4. Mock bank service and verification flow with trust tiers.
5. Rules engine, scheduler and simulated clock.
6. Alerts, escalation and the "my day" queue.
7. Renewal chain and instrument passport.
8. Release workflow with conditions and approvals.
9. Dashboard and reports.
10. Audit log hashing and integrity indicator.
11. Seed data and polish.

Cut scope from the end of this list before you cut polish from the start.

12. Seed data

Create about 60 FDRs across a spread of states: healthy, expiring soon, matured, renewed, mismatched with the bank, pending verification, release requested and released. Include a few deliberately tricky cases, such as an amount mismatch, a renewal that arrives late and a release request blocked by an open discrepancy. Use synthetic data only.

13. Demo script (about 6 minutes)

Follow the statement's own test scenarios in order:

1. **New FDR submission.** An institution submits an instrument and documents.
2. **Verification and discrepancy.** The officer runs bank verification, a mismatch is caught and the discrepancy card shows claimed versus confirmed.
3. **Validity tracking.** Show the dashboard with active, expiring and matured counts.
4. **Approaching maturity.** Fast-forward the clock, reminders fire, the "my day" queue reorders.
5. **Escalation.** An unresolved case escalates up the chain.

6. **Renewal.** The institution uploads a replacement, the renewal chain links old and new, and the passport shows the history.
7. **Authorized release.** A release request passes its conditions checklist and approval chain. Show it blocked once while a discrepancy is open, then unblocked.
8. **Audit.** Show the audit trail and the integrity indicator.

Close with a slide that maps all ten expected-solution items to features.

14. Proving measurable improvement

The statement asks for measurable gains. Run a simple before-and-after comparison on your seeded data, clearly labelled as simulated:

- Steps and time per case under a manual spreadsheet-and-email process versus the portal.
- Days to detect a lapsing FDR.
- Share of lapsing FDRs caught before expiry.
- Time from submission to verification outcome.
- Records with missing or inconsistent fields detected automatically.

Present these as illustrative figures from a stated simulation, not real-world claims.

15. Evaluation criteria checklist

Before submitting, confirm each of these is visible in the demo or deck:

- All ten expected capabilities mapped to features
- All six named test scenarios demonstrated
- Institution-submitted versus bank-confirmed distinction visible
- Configurable reminders and escalation
- Complete history and audit trail
- Role-based access shown with at least two roles
- Measurable improvement slide
- Integration approach described (mock now, authorized bank and AICTE systems later)
- Security approach described

16. Risks and how to handle them

Risk	Mitigation
Real bank verification is not available	Mock it, label it clearly, and show the integration contract
Overbuilding and an unfinished demo	Lock the scope after step 8 of the build order and spend the rest on polish
Looking like a generic CRUD app	Lead the demo with trust tiers and the fast-forward clock
Unsupported claims	Label all metrics as simulated and keep wording modest
Software appearing to replace officials	State that decisions stay with the competent authority and the portal supports them

17. Pitch angle

Open with the problem in one sentence: *AICTE holds crores in deposits, and today nobody can say at a glance which are proven, which are about to lapse and who has to act.* Then show the two ideas, trust tiers and the clock, and let the demo carry the rest. Close with the mapping slide and the improvement numbers.

18. Things to confirm

- Whether SIH rules allow one team to submit more than one problem statement.
- The format of the final judging (live demo, video, deck or a mix) so the demo script can be adjusted.
- Any stack or hosting requirements from the organisers.